MODULE 10 · DOMAIN 2 - DATA PREPARATION

# Into Delta, and measuring retrieval

The table an index needs underneath it, and the numbers that tell you whether retrieval works.

SECTION 2 of 6 EXAM WEIGHT 14% MODULE 10 of 40

admin@sthithapragna.com

An index is only as good as the table beneath it. **And a chunking choice is only as good as the numbers that prove it.**


admin@sthithapragna.com

# From parsed documents to a chunk table

01
**Parsed**
One row per document, full text.

›

02
**Split**
A function returns an array of chunks.

›

03
**Explode**
One row per chunk, with its position.

›

04
**Identify**
Stable chunk ID, plus metadata.

›

05
**Write**
Delta table in Unity Catalog, change feed on.

ON THE EXAM

The shape the index needs is one row per chunk, each with its own unique, stable ID.

# What a chunk table needs

| COLUMN | PURPOSE |
|---|---|
| `chunk_id` | Primary key. Unique and stable, such as doc_id plus position |
| `doc_id` | Which source document the chunk came from |
| `chunk_text` | The text that gets embedded and returned |
| `title, section, page` | Metadata for citations and query-time filters |
| `parent_text` | Optional. The larger section, for parent-child retrieval |
| `updated_at` | When the source last changed |

ON THE EXAM

A Delta Sync index needs a primary key and a text or vector column. Everything else is there to make retrieval better.

# An array per document, or a row per chunk?

ONE ROW PER DOCUMENT

## Chunks in an array column

- Looks tidy
- The index sees one row
- One vector per document
- Chunking has been undone

ONE ROW PER CHUNK

## Flattened with explode

- The index sees every chunk
- One vector per chunk
- Each chunk has its own ID
- Retrieval returns passages

THE TIEBREAKER

An index embeds rows. If the chunks are still inside an array, the index cannot see them as chunks. Flatten before you index.

# The few lines that matter

| STEP | CODE |
| --- | --- |
| Split | chunk_udf(col("text")) returns an array of strings |
| Flatten | posexplode(...) gives one row per chunk, with its position |
| Stable ID | concat_ws("_", col("doc_id"), col("pos")) |
| Write | saveAsTable("main.rag.policy_chunks"), a three-level name |
| Change feed | delta.enableChangeDataFeed = true |

ON THE EXAM

posexplode, not explode, when you want the position. The position is what makes the ID stable across runs.

# What the table gives the index

- A **Delta Sync index** reads a Delta table and keeps itself up to date from it.
- Standard endpoints need **change data feed** on the source. Row tracking provides it automatically.
- Unity Catalog adds **permissions and lineage**, from source file to chunk to index.
- Updates use **MERGE** on the chunk ID, so changed chunks replace old ones.
- The column name **_id** is reserved. Do not use it for your key.

> ON THE EXAM
>
> Change data feed is what lets the index sync only what changed, instead of re-embedding everything.

# What happens on the second run

A NEW RANDOM ID EACH RUN

## Every run looks new

- Appends create duplicates
- Old versions never replaced
- Index keeps growing
- Same passage returned several times

A STABLE ID FROM DOC AND POSITION

## Every run updates

- MERGE matches existing rows
- Changed chunks replace old ones
- Index stays the right size
- Deleted chunks can be removed

THE TIEBREAKER

A UUID generated at write time is unique but not stable. The index needs both.

# The classic metrics

| METRIC | ASKS |
|---|---|
| Precision at k | Of the k chunks returned, what fraction were relevant? |
| Recall at k | Of all the relevant chunks, what fraction were in the top k? |
| Hit rate at k | Was at least one relevant chunk in the top k? |
| MRR | How high did the first relevant chunk appear, on average? |
| NDCG at k | Were the relevant chunks ranked near the top? |

ON THE EXAM

All five need ground truth: for each test question, the IDs of the chunks that should come back. Stable IDs make that possible.

# One question, scored

| ITEM | VALUE |
|---|---|
| Relevant chunks for this question | A and B |
| Retrieved top 5, in order | X, A, Y, Z, W |
| Precision at 5 | 1 relevant of 5 returned = 0.2 |
| Recall at 5 | 1 of 2 relevant found = 0.5 |
| Reciprocal rank | First relevant at position 2 = 1/2 = 0.5 |
| Hit at 5 | Yes |

ON THE EXAM

B was never retrieved. If B holds half the answer, no LLM and no re-ranker can recover it. That is why recall comes first.

# Recall or precision?

RECALL FIRST WHEN

## Missing a chunk is fatal

- Answers need several facts
- Missing one breaks the answer
- Re-ranking can fix order later
- The usual case for rag

PRECISION MATTERS WHEN

## Noise misleads

- Very few chunks fit the prompt
- Irrelevant chunks confuse the LLM
- Cost per token matters
- After recall is already good

THE TIEBREAKER

Low recall is a retrieval problem. Low precision with good recall is an ordering problem, which is exactly what re-ranking fixes.


admin@sthithapragna.com

# MLflow's retrieval judges

| JUDGE | ASKS | NEEDS GROUND TRUTH? |
|---|---|---|
| RetrievalRelevance | Is each retrieved chunk relevant to the request? | No |
| RetrievalSufficiency | Do the chunks contain the expected facts? | Yes, expected facts |
| RetrievalGroundedness | Is the response supported by the chunks? | No |

ON THE EXAM

All three read the retrieved chunks from a trace span of type RETRIEVER. No retriever span, no retrieval judgement.

# A retrieval problem, or a generation problem?

RETRIEVAL PROBLEM

## The facts never arrived

- Sufficiency fails
- Relevance scores low
- Fix chunking, sources, recall
- Sections 2 and 4

GENERATION PROBLEM

## The facts arrived, and were ignored

- Sufficiency passes
- Groundedness fails
- Fix the prompt, model or guardrails
- Section 3

THE TIEBREAKER

Sufficient but not grounded means the model had what it needed and went beyond it. Do not re-chunk: fix generation.


admin@sthithapragna.com

# Evaluate every retrieval change the same way

01

### Fix the set

Same questions and ground truth every time.

02

### Change one thing

Chunk size, embedding model, or reranker.

03

### Score

Recall, precision, and the retrieval judges.

04

### Log

Each variant as an MLflow run.

05

### Keep

The variant that wins on the metric that matters.

ON THE EXAM

One change at a time, against one fixed evaluation set. Otherwise you cannot tell which change helped.

> An engineer's chunking function returns an array of chunks for each document. The engineer writes the result to a Delta table with one row per document and an array column, then creates a Delta Sync index on it.
>
> Retrieval returns whole documents rather than passages, and quality is poor.

## What should the engineer change?

A Use a larger embedding model with more dimensions per vector

B Store the array as a JSON string column in the same row instead

C Add a reranker to each query so the best document ranks first

D Flatten to one row per chunk, each with a unique chunk ID key

◆ PAUSE AND DECIDE

## What should the engineer change?

A Larger embedding model

*Still one vector per document. The chunking is still undone.*

B Array as JSON string

*Same one row per document, in a different type.*

C Reranker

*Reorders whole documents. Still no passages.*

D One row per chunk

Each chunk becomes its own row, key and vector.

THE SIGNAL

Returns whole documents rather than passages. The index can only return what a row contains.

A nightly job re-chunks all policy documents and appends the results to the chunk table, assigning each chunk a new UUID.

After a month, the index has grown fourfold, and the same passage often appears several times in one result set.

## What is the best fix?

A Retrieve more chunks per query to make room for the duplicates

B Enable the reranker so that the best copy always ranks first

C Stable IDs from document and position, and MERGE, not append

D Turn off change data feed on the chunk table to stop growth

◆ PAUSE AND DECIDE

## What is the best fix?

A Retrieve more

*More room for the same duplicates.*

B Reranker

*Ranks duplicates highly together. Does not remove them.*

C Stable IDs and MERGE

Each run updates existing rows instead of adding copies.

D Turn off change feed

*Breaks index sync on standard endpoints. Duplicates remain.*

THE SIGNAL

A new UUID each night plus append means every run adds a full copy. Unique is not enough. IDs must be stable.

A team is comparing two chunking strategies. It has 200 test questions, each labelled with the IDs of the chunks that contain its answer.

The team wants to know which strategy more often gets the needed chunks into the top 5 results.

## Which metric should the team compare?

A Recall at 5, against the labelled chunk IDs

B BLEU score of the final answers against references

C Perplexity of the LLM on the test questions

D The Safety judge, run on every test question

◆ PAUSE AND DECIDE

# Which metric should the team compare?

**A** Recall at 5

Exactly the question: were the needed chunks in the top 5?

**B** BLEU

*Scores word overlap of answers. Mixes retrieval with generation.*

**C** Perplexity

*About the language model's fluency. Says nothing about retrieval.*

**D** Safety judge

*Checks for harmful content. A different question entirely.*

THE SIGNAL

Labelled chunk IDs, and in the top 5. Ground truth plus a cut-off means precision or recall at k. Needed chunks means recall.

> An engineer evaluates a rag assistant with MLflow judges. RetrievalSufficiency passes for most questions.
>
> RetrievalGroundedness fails often: answers include claims that do not appear in the retrieved chunks.

## Where should the engineer focus next?

A Retrieval: change the chunking strategy and re-index

B Generation: the prompt, the model or output guardrails

C Sources: add more documents to the index for coverage

D Ranking: enable the reranker so better chunks come first

◆ PAUSE AND DECIDE

## Where should the engineer focus next?

A **Change chunking**
*Retrieval is already sufficient. Nothing to fix there.*

B **The generation step**
The facts arrived. The model went beyond them.

C **More documents**
*More material to wander from. Retrieval already has enough.*

D **Reranker**
*Reorders chunks that were already sufficient.*

THE SIGNAL

Sufficient but not grounded. The model had what it needed, and still added claims. That is a generation problem.

# What to carry forward

- **One row per chunk**, flattened with posexplode, with a unique and **stable** chunk ID.
- Delta table in Unity Catalog, **change data feed on**, updated with MERGE.
- Precision, recall, hit rate, MRR, NDCG at k: all need labelled chunk IDs.
- MLflow judges: **Relevance** and **Groundedness** need no ground truth; **Sufficiency** does.
- Sufficient but not grounded is a generation problem. Do not re-chunk.

> ON THE EXAM
>
> That completes section 2. Next, section 3, the largest: module 11 on choosing an LLM.